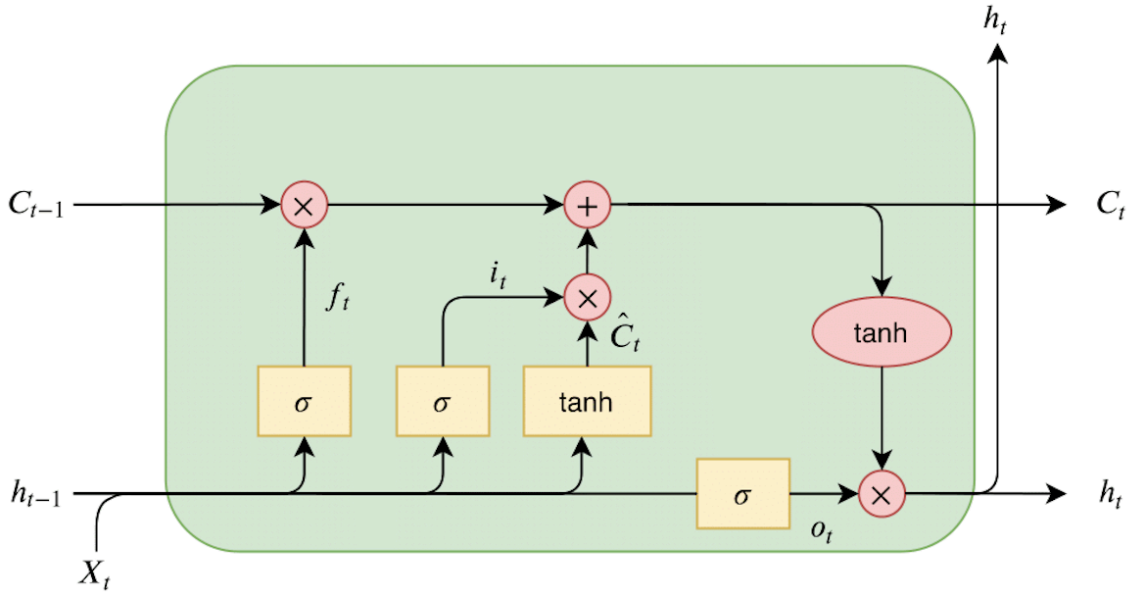
Lab 10 Sequence

Section 1: Math



Given the following values:

- Previous cell state $C_{t-1} = 2$
- Previous hidden state $h_{t-1} = 1$
- Current input $x_t = 0.5$

And the following weights and biases for each gate:

1. Forget gate f_t:

- Weights: $W_{xf} = 1$, $W_{hf} = 1.5$
- Bias: $b_f = -0.5$

2. Input gate i_t:

- Weights: $W_{xi} = -1$, $W_{hi} = 1$
- Bias: $b_i = 0.5$

3. Candidate cell state $\hat{C}_t$:

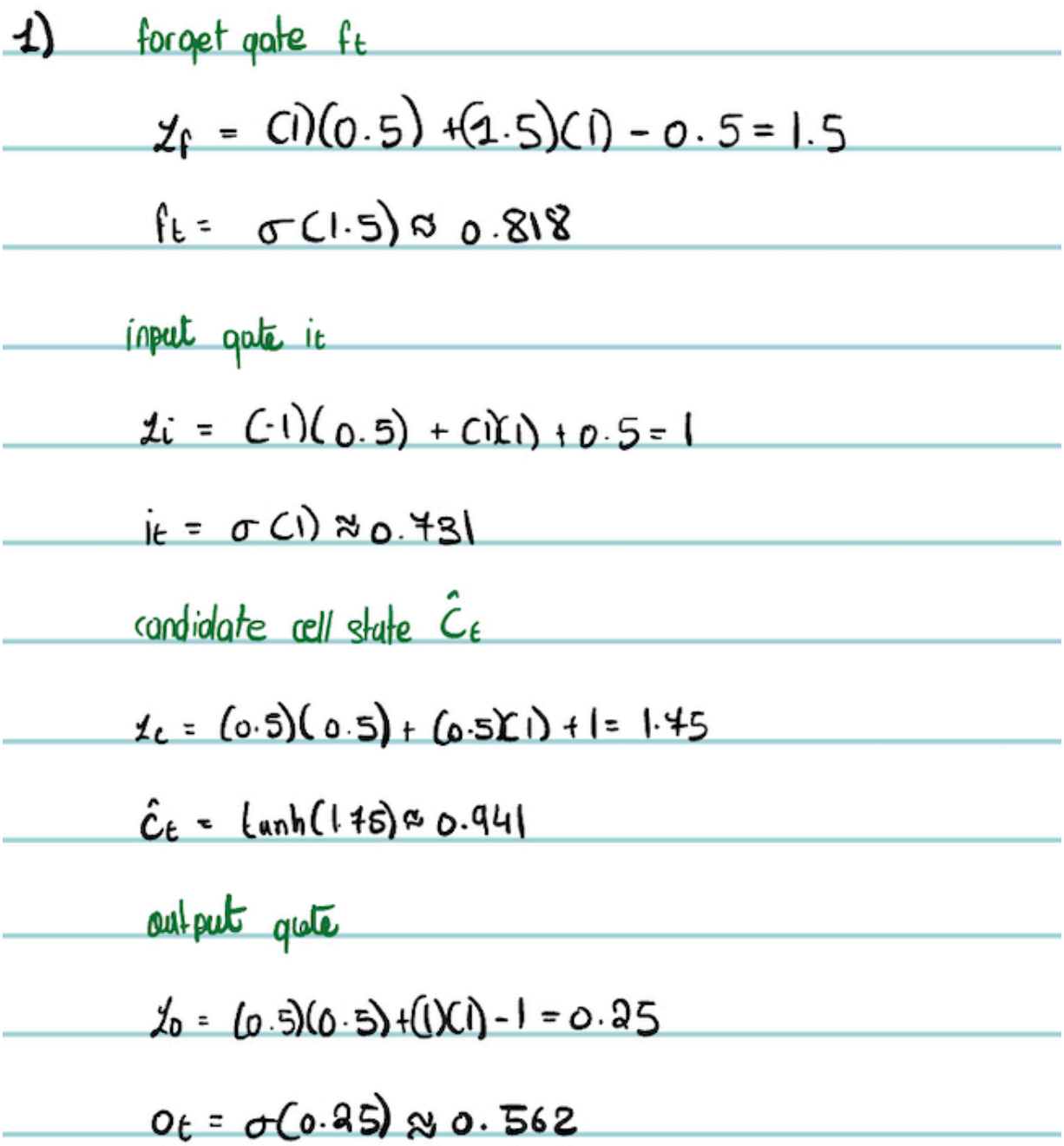
- Weights: $W_{xc} = 0.5$, $W_{hc} = 0.5$
- Bias: $b_c = 1$

4. Output gate o_t:

- Weights: $W_{xo} = 0.5$, $W_{ho} = 1$
- Bias: $b_o = -1$

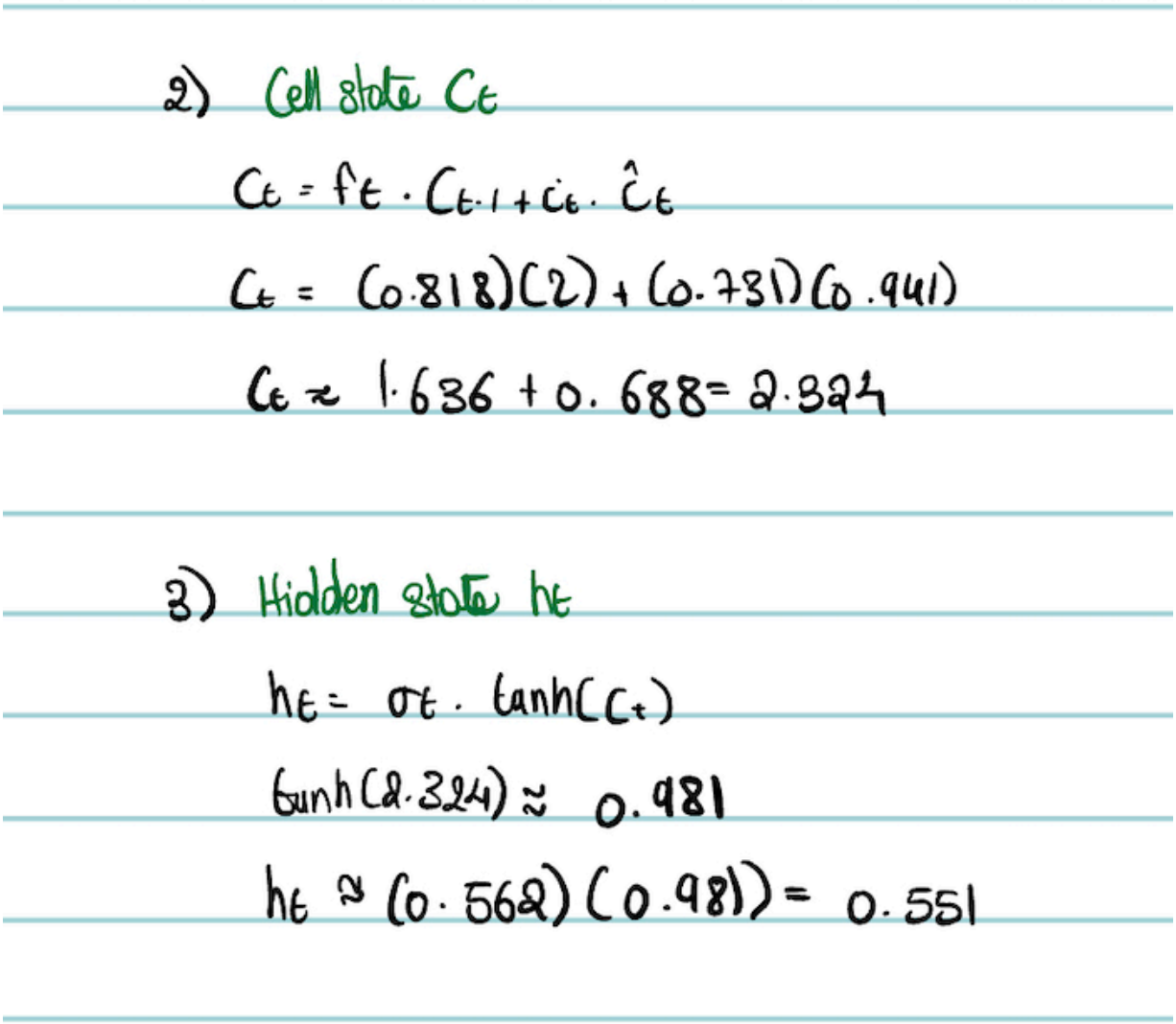
❓ 1.1 Calculate the forget gate f_t, input gate i_t, candidate cell state $\hat{C}_t$, and output gate values o_t.

In [56]:
```
from IPython.display import Image, display
display(Image("1.1.png"))
```



❓ 1.2 Use these values to calculate the new cell state C_t and hidden state h_t.

In [57]:
```
from IPython.display import Image, display
display(Image("1.2.png"))
```



Section 2: Featurize a SMILES string

In the lecture, we discussed methods for translating implicit, text-based representations (e.g., SMILES for small molecules) into numerical data suitable for machine learning models. Now, let's put this theory into practice!

A little background on the data we'll be working with today: this dataset is part of a benchmarking collection known as **Delaney**, which contains aqueous solubility data for documented molecules, expressed as log solubility in moles per liter.

2.1 SMILES Tokenization and Numericalization

In this section, we will preprocess SMILES strings into a format that can be fed to sequence models, such as 1D convolutional neural networks and transformers, and enables these models to learn their own representations of molecular properties.

To prepare SMILES strings for a sequence model, we break them down into lists of substrings (called tokens) and turn them into lists of integer values (numericalization).

❓ 2.1.1 Print the unique characters from all SMILES in the dataset (provided as `sol_data.csv`).

In [58]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
#from google.colab import files
'''
uploaded = files.upload()

for fn in uploaded.keys():
  print('User uploaded file "{name}" with length {length} bytes'.format(
      name=fn, length=len(uploaded[fn])))
'''
df = pd.read_csv("sol_data.csv")
```

In [59]:
```
# please write your code below
smiles_list = df["smiles"]

# Get unique characters
unique_chars = set()

for smi in smiles_list:
    unique_chars.update(list(smi))

# Print result
print(sorted(unique_chars))
```

```
['#', '(', ')', '/', '1', '2', '3', '4', '5', '6', '7', '8', '=', 'B', 'C',
'F', 'H', 'I', 'N', 'O', 'P', 'S', '[', '\\', ']', 'c', 'l', 'n', 'o', 'r',
's']
```

❓ 2.1.2 Should we directly use this set of characters as our vocabulary?

✅ Answer: No beause they are just raw set of integers with no inherent meaning for the model to learn from.

To address the issue of undesired character splitting, we need to manually define a search pattern for letters that represent atom types.

In [60]:
```
%%capture
!pip install rdkit-pypi
# If rdkit-pypi gives some _ARRAY_API error, uncomment the next line
!pip install numpy==1.26.4
from rdkit import Chem
```

In [61]:
```
# first let's take a look at what unique atom types are present
all_atom_type = [atom.GetSymbol() for smiles in smiles_list for atom in Chem
all_atom_type = list(set(all_atom_type))
all_atom_type.sort()
print(all_atom_type)
```

```
['Br', 'C', 'Cl', 'F', 'I', 'N', 'O', 'P', 'S']
```

In [62]:
```
# then let's check what "letter" and "non-letter" characters are there in al
letter_elements = [char for char in unique_chars if char.isalpha()]
non_letter_elements = [char for char in unique_chars if not char.isalpha()]
letter_elements.sort()
non_letter_elements.sort()

print(letter_elements)
print(non_letter_elements)
```

```
['B', 'C', 'F', 'H', 'I', 'N', 'O', 'P', 'S', 'c', 'l', 'n', 'o', 'r', 's']
['#', '(', ')', '/', '1', '2', '3', '4', '5', '6', '7', '8', '=', '[', '\\',
']']
```

In [63]:
```
# now, let's manually define the search pattern, taking into account the imp
atom_pattern = ['Br', 'Cl', 'C', 'c', 'F', 'I', 'N', 'n', 'O', 'o', 'S', 's'
token_list = atom_pattern+non_letter_elements
print(token_list)
```

```
['Br', 'Cl', 'C', 'c', 'F', 'I', 'N', 'n', 'O', 'o', 'S', 's', 'H', '#',
'(', ')', '/', '1', '2', '3', '4', '5', '6', '7', '8', '=', '[', '\\', ']']
```

We're almost there! The next step is to map each of these tokens to a unique integer, transforming each SMILES string into a distinct feature vector for model training.

In [64]:
```
# define the tokenization function
import re
def tokenize_smiles(smiles, token_list=token_list):
  pattern = r'|'.join(re.escape(token) for token in token_list)
  output = re.findall(pattern, smiles)
  return output

num_dict = {token: idx for idx, token in enumerate(token_list)}
print(num_dict)
```

```
{'Br': 0, 'Cl': 1, 'C': 2, 'c': 3, 'F': 4, 'I': 5, 'N': 6, 'n': 7, 'O': 8,
'o': 9, 'S': 10, 's': 11, 'H': 12, '#': 13, '(': 14, ')': 15, '/': 16, '1':
17, '2': 18, '3': 19, '4': 20, '5': 21, '6': 22, '7': 23, '8': 24, '=': 25,
'[': 26, '\\': 27, ']': 28}
```

❓ 2.1.3 Given the functions above, please print the results of the tokenization and numericalization for the first two SMILES strings in the dataset.

In [65]:
```
# please write your code below
first_two_smiles = smiles_list.iloc[:2]

for smi in first_two_smiles:
    tokens = tokenize_smiles(smi)
    numbers = [num_dict[token] for token in tokens]

    print("SMILES:", smi)
    print("Tokens:", tokens)
    print("Numbers:", numbers)
    print()
```

```
SMILES: CCCCN(C)C(=O)Nc1ccc(Cl)c(Cl)c1
Tokens: ['C', 'C', 'C', 'C', 'N', '(', 'C', ')', 'C', '(', '=', 'O', ')',
'N', 'c', '1', 'c', 'c', 'c', '(', 'Cl', ')', 'c', '(', 'Cl', ')', 'c', '1']
Numbers: [2, 2, 2, 2, 6, 14, 2, 15, 2, 14, 25, 8, 15, 6, 3, 17, 3, 3, 3, 14,
1, 15, 3, 14, 1, 15, 3, 17]

SMILES: CN(C)C(=O)SCCCCOc1ccccc1
Tokens: ['C', 'N', '(', 'C', ')', 'C', '(', '=', 'O', ')', 'S', 'C', 'C',
'C', 'C', 'O', 'c', '1', 'c', 'c', 'c', 'c', 'c', '1']
Numbers: [2, 6, 14, 2, 15, 2, 14, 25, 8, 15, 10, 2, 2, 2, 2, 8, 3, 17, 3, 3,
3, 3, 3, 17]
```

❓ 2.1.4 Do you believe these resulting feature vectors can be directly fed into your model?

✅ Answer: No because the feature vectors have different lenghts, do not include start/end context.

To ensure that all feature vectors have the same length, we can append zeros up to a fixed value, a process known as padding. The fixed length can be determined by the longest string found in our dataset.

Additionally, you may want to update **num_dict**, as 0 is now mapped to 'Br'. To represent meaningful tokens, increment each numerical value in the mapping dictionary by one.

❓ 2.1.5 Please print out the **fixed length**, the updated **num_dict**, and the **feature vector** of the first SMILES string in the dataset.

- Note that the `tokenize_smiles()` function from earlier calls `num_dict` by default. Updating `num_dict` here will change the results of some code blocks above if you were to rerun them. If you make a new dictionary called something other than `num_dict`, keep in mind the `tokenize_smiles()` function *will continue to call the old version of* `num_dict` *by default, since the dictionary named* `num_dict` *is a default input!* If you make a new dictionary, be careful using provided code below and make sure to call your new dictionary explicitly.

In [66]:
```python
# please write your code below
# Step 1: tokenize ALL SMILES
tokenized_smiles = [tokenize_smiles(smi) for smi in smiles_list]

# Step 2: compute fixed length
fixed_length = max(len(tokens) for tokens in tokenized_smiles)

print("Fixed length:", fixed_length)

# Step 3: update num_dict (shift by +1)
updated_num_dict = {token: idx + 1 for token, idx in num_dict.items()}

print("Updated num_dict:")
print(updated_num_dict)

# Step 4: get first SMILES
first_smi = smiles_list.iloc[0]
tokens = tokenize_smiles(first_smi)

# Step 5: convert to numbers
feature_vector = [updated_num_dict[token] for token in tokens]

# Step 6: pad with 0s
padded_vector = feature_vector + [0] * (fixed_length - len(feature_vector))

print("\nFirst SMILES:", first_smi)
print("Tokens:", tokens)
print("Feature vector (padded):", padded_vector)
```

```
Fixed length: 97
Updated num_dict:
{'Br': 1, 'Cl': 2, 'C': 3, 'c': 4, 'F': 5, 'I': 6, 'N': 7, 'n': 8, 'O': 9,
'o': 10, 'S': 11, 's': 12, 'H': 13, '#': 14, '(': 15, ')': 16, '/': 17, '1':
18, '2': 19, '3': 20, '4': 21, '5': 22, '6': 23, '7': 24, '8': 25, '=': 26,
'[': 27, '\\': 28, ']': 29}

First SMILES: CCCCN(C)C(=O)Nc1ccc(Cl)c(Cl)c1
Tokens: ['C', 'C', 'C', 'C', 'N', '(', 'C', ')', 'C', '(', '=', 'O', ')',
'N', 'c', '1', 'c', 'c', 'c', '(', 'Cl', ')', 'c', '(', 'Cl', ')', 'c', '1']
Feature vector (padded): [3, 3, 3, 3, 7, 15, 3, 16, 3, 15, 26, 9, 16, 7, 4,
18, 4, 4, 4, 15, 2, 16, 4, 15, 2, 16, 4, 18, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0]
```

2.2: Model evaluation

Using the provided dataset, implement a Random Forest model (keeping all hyperparameters at their default values) to compare model performance when molecules are featurized using MACCS keys, RDKit descriptors and the tokenization and numericalization methods you just completed.

> Note: This question has a lot of moving parts, but you've done all of them before! This is the way we've featurized SMILES strings in the past.

❓ 2.2.1 Please report the model performance by providing the R^2, RMSE, and MAE metrics for each featurization method.

In [67]:
```
from rdkit.Chem import MACCSkeys
from rdkit.Chem import Descriptors
import math
# You may need to use a different version of numpy- numpy==2.0.0 worked for
!pip install numpy==1.26.4
import numpy as np
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute
from sklearn.ensemble import RandomForestRegressor
```

```
Requirement already satisfied: numpy==1.26.4 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (1.26.4)
```

In [68]:
```
# please write your code below
#Using the provided dataset, implement a Random Forest model (keeping all hy
from rdkit import Chem
from rdkit.Chem import MACCSkeys, Descriptors
import numpy as np
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute
from sklearn.ensemble import RandomForestRegressor


# 1. Get data
smiles_list = df["smiles"]
y = df["y"]
split = df["split"]

# 2. MACCS features

X_maccs = []
for smi in smiles_list:
    mol = Chem.MolFromSmiles(smi)
    fp = MACCSkeys.GenMACCSKeys(mol)
    X_maccs.append(list(fp))

X_maccs = np.array(X_maccs)


# 3. RDKit descriptors

X_desc = []
for smi in smiles_list:
    mol = Chem.MolFromSmiles(smi)
    vals = [func(mol) for name, func in Descriptors.descList]
    X_desc.append(vals)

X_desc = np.array(X_desc)
```

```
# 4. Tokenized + padded features

tokenized_smiles = [tokenize_smiles(smi) for smi in smiles_list]
fixed_length = max(len(tokens) for tokens in tokenized_smiles)

updated_num_dict = {token: idx + 1 for token, idx in num_dict.items()}

X_token = []
for tokens in tokenized_smiles:
    nums = [updated_num_dict[token] for token in tokens]
    padded = nums + [0] * (fixed_length - len(nums))
    X_token.append(padded)

X_token = np.array(X_token)


# 5. Train/test split (USE YOUR COLUMN)

train_idx = split == "train"
test_idx = split == "test"

def evaluate(X):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    model = RandomForestRegressor()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    r2 = r2_score(y_test, y_pred)
    rmse = root_mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)

    return r2, rmse, mae


# 6. Run all
print("MACCS:", evaluate(X_maccs))
print("Descriptors:", evaluate(X_desc))
print("Tokenized:", evaluate(X_token))
```

```
MACCS: (0.8203600210495412, 0.4496442390464399, 0.3212281017042182)
Descriptors: (0.9178457936615811, 0.3040762060633957, 0.21015697466452285)
Tokenized: (0.6936645696210011, 0.5871730992350819, 0.45397845650564533)
```

❓ 2.2.2 Rank the model performances from highest to lowest and provide some reasons for the observed rankings.

✅ Answer:Descriptors perform best, followed by MACCS, and then Tokenized performs worst. Descriptors use continuous, chemically meaningful features and achieve an R-squared of about 0.918 with RMSE around 0.304 and MAE around 0.210, MACCS uses binary structural fingerprints with an R-squared of about 0.823, RMSE around 0.450, and MAE around 0.321, while Tokenized uses sequence-based representations that lose

structural detail, resulting in the lowest R-squared of about 0.694 and the highest errors, with RMSE around 0.587 and MAE around 0.454.

Section 3: Generate structures using RNN

In this section, we will use a Recurrent Neural Network (RNN) to generate SMILES strings. We will employ Keras to build our own RNN model from scratch.

Before the model can generate a SMILES string, it must learn to predict the next character in the sequence based on the current character.

We have previously created and updated a dictionary, **num_dict**, that assigns each token a unique integer identifier. When the model generates numerical values, we need a mechanism to map these integers back to valid characters in a SMILES string.

To achieve this, let's create two dictionaries, **char_to_int** and **int_to_char**, that facilitate bi-directional mapping between characters/tokens and integers. Additionally, we will add a special token "E" (for End) and map it to 0 to maintain consistency and accommodate the padding mechanism.

❓ 3.1 Please print out the dictionaries **char_to_int** and **int_to_char**, ensuring that the pair "E: 0" or "0: E" is placed at the front.

In [69]:
```
# please write your code below
# token -> integer
char_to_int = {"E": 0}
for token, idx in updated_num_dict.items():
    char_to_int[token] = idx

# integer -> token
int_to_char = {idx: token for token, idx in char_to_int.items()}

print("char_to_int:")
print(char_to_int)
print()

print("int_to_char:")
print(int_to_char)
```

```
char_to_int:
{'E': 0, 'Br': 1, 'Cl': 2, 'C': 3, 'c': 4, 'F': 5, 'I': 6, 'N': 7, 'n': 8,
'O': 9, 'o': 10, 'S': 11, 's': 12, 'H': 13, '#': 14, '(': 15, ')': 16, '/':
17, '1': 18, '2': 19, '3': 20, '4': 21, '5': 22, '6': 23, '7': 24, '8': 25,
'=': 26, '[': 27, '\\': 28, ']': 29}

int_to_char:
{0: 'E', 1: 'Br', 2: 'Cl', 3: 'C', 4: 'c', 5: 'F', 6: 'I', 7: 'N', 8: 'n',
9: 'O', 10: 'o', 11: 'S', 12: 's', 13: 'H', 14: '#', 15: '(', 16: ')', 17:
'/', 18: '1', 19: '2', 20: '3', 21: '4', 22: '5', 23: '6', 24: '7', 25: '8',
26: '=', 27: '[', 28: '\\', 29: ']'}
```

As mentioned, the objective of model training is to learn from the current character and predict the subsequent adjacent character.

In [70]:
```python
def gen_data(data, int_to_char, char_to_int, embed):
    # Initialize the one-hot array (embed refers to max length of SMILES)
    one_hot = np.zeros((data.shape[0], embed, len(char_to_int)), dtype=np.ir

    for i, smile in enumerate(data):
        # Tokenize the SMILES string
        tokens = tokenize_smiles(smile)

        # Encode the characters, ensuring we do not exceed embed
        for j, token in enumerate(tokens[:embed]):  # Limit to `embed`
            one_hot[i, j, char_to_int[token]] = 1

        # Fill remaining positions with the end character "E" if there are f
        if len(tokens) < embed:
            one_hot[i, len(tokens):, char_to_int["E"]] = 1  # Fill with "E"
        else:
            # If tokens fill the array, we set "E" only in the last position
            one_hot[i, embed-1, char_to_int["E"]] = 1

    # Return two arrays: one for input and one for output
    return one_hot[:, :-1, :], one_hot[:, 1:, :]
```

In [71]:
```python
# get longest sequence
from sklearn.utils import shuffle
embed = max(len(tokens) for tokens in tokenized_smiles)

# Get datasets
X, Y = gen_data(df['smiles'], int_to_char, char_to_int, embed)
X, Y = shuffle(X, Y)
```

In [72]:
```python
Y.shape
```

Out[72]:
```
(1128, 96, 30)
```

❓ 3.2 Explain the shape of Y.

✅ Answer: 1128 smiles, 96 is max len a seuquence can have, 30 possible tokens

Now it's time to construct our own RNN! The first steps of network construction have been done for you - read them carefully.

Some documentation links corresponding to the dependencies loaded below:

- Model
- Sequential
  - This lets us group a linear stack of layers into a model. Later in this section you'll call methods associated with the object:
    - Compiling the model. This step adds important hyperparameters like the learning rate, optimizer, and loss function.
    - Fitting the model
    - Making predictions
- LSTM
  - Adds an LSTM layer to the network! `return_sequences = True` tells the layer to return not just the last output in the sequence, but the full sequnce.
- Dropout
  - Dropout adds a layer that helps prevent overfitting by randomly setting input weights to 0 with some frequency during training. This obviously prevents the neurons with 0 inputs from contributing to the forward pass, but it also prevents these neurons from contributing to the backward pass. *Why might this help prevent overfitting?*
- Dense
  - Fully connected layers (like everything in the previous lab) are sometimes called "dense" layers.
- Example

In [73]:
```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /Users/dianadayekh/anaconda3/en
vs/clean_cluster/lib/python3.11/site-packages (2.21.0)
Requirement already satisfied: absl-py>=1.0.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (2.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /Users/dianadayekh/anaco
nda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (1.6.
3)
Requirement already satisfied: flatbuffers>=25.9.23 in /Users/dianadayekh/an
aconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (2
5.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /User
s/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (fro
m tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (0.
2.0)
Requirement already satisfied: libclang>=13.0.0 in /Users/dianadayekh/anacon
da3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (18.1.
1)
Requirement already satisfied: opt_einsum>=2.3.2 in /Users/dianadayekh/anaco
nda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (3.4.
0)
Requirement already satisfied: packaging in /Users/dianadayekh/anaconda3/env
s/clean_cluster/lib/python3.11/site-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflo
w) (7.34.1)
Requirement already satisfied: requests<3,>=2.21.0 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (2.
32.5)
Requirement already satisfied: setuptools in /Users/dianadayekh/anaconda3/en
vs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (80.10.2)
Requirement already satisfied: six>=1.12.0 in /Users/dianadayekh/anaconda3/e
nvs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /Users/dianadayekh/anacon
da3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (3.3.
0)
Requirement already satisfied: typing_extensions>=3.6.6 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflo
w) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /Users/dianadayekh/ana
conda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (1.
78.0)
Requirement already satisfied: keras>=3.12.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /Users/dianadayekh/anaconda
3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow) (1.26.4)
Requirement already satisfied: h5py<3.15.0,>=3.11.0 in /Users/dianadayekh/an
aconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflow)
(3.14.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from tensorflo
w) (0.5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /Users/dianadayek
```

```
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<
3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /Users/dianadayekh/anaconda3/
envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.21.0->t
ensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.2
1.0->tensorflow) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from requests<3,>=2.2
1.0->tensorflow) (2026.1.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /Users/dianadayekh/anac
onda3/envs/clean_cluster/lib/python3.11/site-packages (from astunparse>=1.6.
0->tensorflow) (0.46.3)
Requirement already satisfied: rich in /Users/dianadayekh/anaconda3/envs/cle
an_cluster/lib/python3.11/site-packages (from keras>=3.12.0->tensorflow) (1
4.3.3)
Requirement already satisfied: namex in /Users/dianadayekh/anaconda3/envs/cl
ean_cluster/lib/python3.11/site-packages (from keras>=3.12.0->tensorflow)
(0.1.0)
Requirement already satisfied: optree in /Users/dianadayekh/anaconda3/envs/c
lean_cluster/lib/python3.11/site-packages (from keras>=3.12.0->tensorflow)
(0.19.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /Users/dianadayekh/a
naconda3/envs/clean_cluster/lib/python3.11/site-packages (from rich->keras>=
3.12.0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /Users/dianadayek
h/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from rich->kera
s>=3.12.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /Users/dianadayekh/anaconda3/en
vs/clean_cluster/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->r
ich->keras>=3.12.0->tensorflow) (0.1.2)
```

In [74]:
```python
# load necessary dependencies
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import LSTM
from sklearn.utils import shuffle
```

In [75]:
```python
"""CREATING THE LSTM MODEL """
# Create the model (simple 2 layer LSTM)
model = Sequential() # This indicates that you are creating a linear stack o
# The first dimension of input_shape is set to None, which allows the model
model.add(LSTM(256, input_shape=(None, X.shape[2]), return_sequences=True))
model.add(Dropout(0.25))
```

```
/Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-package
s/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape
`/`input_dim` argument to a layer. When using Sequential models, prefer usin
g an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

❓ 3.3 What is the last line of code in the block above doing? How will that help prevent overfitting?

✅ Answer: It adds a dropout layer after the LSTM. During training, it randomly sets 25% of the layer's outputs to zero at each update. This helps prevent overfitting because it stops the model from relying too heavily on specific neurons or memorizing the training data, encouraging it to learn more general patterns that transfer better to unseen SMILES strings.

❓ 3.4 Please follow the instructions in the codeblock below on finishing the construction of your model.

In [76]:
```
# please write your code below
# add the second LSTM layer of 256 neurons
# there is no need to explicitly set the input_shape as it can be inferred f
model.add(LSTM(256, return_sequences = True))

# add the second dropout layer with the same dropout rate (0.25) as the prev
model.add(Dropout(0.25))

# add the output layer which is a fully connected (dense) layer.
# use "softmax" as the activation function
model.add(Dense(len(char_to_int), activation='softmax'))
```

In [77]:
```
# print out a summary of your model
print(model.summary())
```

Model: "sequential_5"

Layer (type)	Output Shape	Par
lstm_9 (LSTM)	(None, None, 256)	293
dropout_6 (Dropout)	(None, None, 256)	
lstm_10 (LSTM)	(None, None, 256)	525
dropout_7 (Dropout)	(None, None, 256)	
dense_5 (Dense)	(None, None, 30)	7

Total params: 826,910 (3.15 MB)
Trainable params: 826,910 (3.15 MB)
Non-trainable params: 0 (0.00 B)
None

In [78]:
```
# Compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam')

# Fit the model
history = model.fit(X, Y, epochs=10, batch_size=256)
```

```
Epoch 1/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 478ms/step - loss: 2.4149
Epoch 2/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 508ms/step - loss: 1.1038
Epoch 3/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 532ms/step - loss: 0.9677
Epoch 4/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 466ms/step - loss: 0.8125
Epoch 5/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 520ms/step - loss: 0.7632
Epoch 6/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 476ms/step - loss: 0.7176
Epoch 7/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 524ms/step - loss: 0.6862
Epoch 8/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 3s 514ms/step - loss: 0.6670
Epoch 9/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 459ms/step - loss: 0.6499
Epoch 10/10
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 451ms/step - loss: 0.6372
```

Now let's examine how well the trained model can replicate the training data. In this step, we will compare the model's predictions to the actual results by identifying the character indices using **np.argmax** and calculating the differences.

In [79]:
```
"""PREDICTION"""
# Calculate predictions
predictions = model.predict(X, verbose=0)
# Compare to correct result
train_res = np.argmax(Y,axis=2) - np.argmax(predictions,axis=2)
# Count correct and incorrect predictions
no_false = np.count_nonzero(train_res)
no_true = len(Y)*embed-no_false
print("Average success rate on training set: %s %%" %str(np.round(100*no_tru
```

```
Average success rate on training set: 81.71 %
```

❓ 3.5 How should we inteprete the average success rate? (What does this mean?)

✅ Answer: The average success rate represents the percentage of tokens for which the model correctly predicts the next character in the sequence on the training set. An 81.92% success rate means that, on average, the model correctly predicts the next SMILES token about 82% of the time. This indicates that the model has learned many of the patterns in the training data, but still makes errors.

❓ 3.6 Display the first 10 SMILES from X along with their corresponding predicted SMILES.

In [80]:
```
# please write your code below
X_index = np.argmax(X[:10], axis=2)
predictions_index = np.argmax(predictions, axis=2)

def decode(indices, int_to_char):
```

```
    tokens = []
    for idx in indices:
        token = int_to_char[idx]
        if token == 'E':
            break
        tokens.append(token)
    return ''.join(tokens)

for i in range(10):
    true_smiles = decode(X_index[i], int_to_char)
    pred_smiles = decode(predictions_index[i], int_to_char)
    print("True SMILES:", true_smiles)
    print("Predicted SMILES:", pred_smiles)
    print()
```

```
True SMILES: CCC(=O)OC3CCC4C2CCC1=CC(=O)CCC1(C)C2CCC34C
Predicted SMILES:

True SMILES: CCC(=O)OC
Predicted SMILES:

True SMILES: C=Cc1ccccc1
Predicted SMILES:

True SMILES: CCCCCC=C
Predicted SMILES:

True SMILES: CC#C
Predicted SMILES:

True SMILES: Cc2cnc1cncnc1n2
Predicted SMILES:

True SMILES: CCO(=S)(OCC)N2C(=O)c1ccccc1C2=O
Predicted SMILES:

True SMILES: CCCC(C)C1(CC)C(=O)NC(=S)NC1=O
Predicted SMILES:

True SMILES: Nc1cccc(c1)N(=O)=O
Predicted SMILES:

True SMILES: N#Cc1ccccc1
Predicted SMILES:
```

❓ 3.7 Based on the predictions, do you think the average success rate is a reliable measure of prediction accuracy? What are your thoughts on this?

✅ Answer: No, the average success rate does not appear to be a fully reliable measure of prediction quality here. Although the reported success rate is about 81.9%, the decoded predictions are mostly empty or extremely short, which suggests the model is often predicting the end token "E" too early. This likely happens because many positions in the padded training sequences contain "E", so the model can achieve a relatively high

token-level accuracy by correctly predicting padding or end tokens without actually learning to generate full SMILES strings well.

❓ 3.8 We are going to evaluate movie reviews using the built-in IMDB dataset from TensorFlow. LSTM-based text classification will be used to classify reviews into positive or negative

In [81]:
```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense


# Loading the IMDB dataset

VOCAB_SIZE = 10000
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)

print(f"Number of training samples: {len(x_train)}")
print(f"Number of test samples: {len(x_test)}")

# map the  words to integer indices
word_index = imdb.get_word_index()

# Invert the mapping to get integer indices -> words
# Note: The first indices (0, 1, 2) are reserved for special tokens (<PAD>,
index_to_word = {idx: word for word, idx in word_index.items()}

def decode_review(encoded_review):
    """
    Convert a sequence of integer indices into a string of words.
    The IMDB dataset reserves indices 0, 1, 2, 3 for special tokens.
    """
    return " ".join([index_to_word.get(i - 3, "?") for i in encoded_review i
```

```
Number of training samples: 25000
Number of test samples: 25000
```

❓ 3.8.1 Print a few sample reviews

In [82]:
```
# please write your code below
for i in range(3):
    print(f"Review {i+1}:")
    print(decode_review(x_train[i]))
    print(f"Label: {y_train[i]}")
    print("-" * 80)
```

```
Review 1:
this film was just brilliant casting location scenery story direction everyo
ne's really suited the part they played and you could just imagine being the
re robert is an amazing actor and now the same being director father came fr
om the same scottish island as myself so i loved the fact there was a real c
onnection with this film the witty remarks throughout the film were great it
was just brilliant so much that i bought the film as soon as it was released
for and would recommend it to everyone to watch and the fly fishing was amaz
ing really cried at the end it was so sad and you know what they say if you
cry at a film it must have been good and this definitely was also to the two
little boy's that played the of norman and paul they were just brilliant chi
ldren are often left out of the list i think because the stars that play the
m all grown up are such a big profile for the whole film but these children
are amazing and should be praised for what they have done don't you think th
e whole story was so lovely because it was true and was someone's life after
all that was shared with us all
Label: 1
--------------------------------------------------------------------------------
----
Review 2:
big hair big boobs bad music and a giant safety pin these are the words to b
est describe this terrible movie i love cheesy horror movies and i've seen h
undreds but this had got to be on of the worst ever made the plot is paper t
hin and ridiculous the acting is an abomination the script is completely lau
ghable the best is the end showdown with the cop and how he worked out who t
he killer is it's just so damn terribly written the clothes are sickening an
d funny in equal the hair is big lots of boobs men wear those cut shirts tha
t show off their sickening that men actually wore them and the music is just
trash that plays over and over again in almost every scene there is trashy m
usic boobs and taking away bodies and the gym still doesn't close for all jo
king aside this is a truly bad film whose only charm is to look back on the
disaster that was the 80's and have a good old laugh at how bad everything w
as back then
Label: 0
--------------------------------------------------------------------------------
----
Review 3:
this has to be one of the worst films of the 1990s when my friends i were wa
tching this film being the target audience it was aimed at we just sat watch
ed the first half an hour with our jaws touching the floor at how bad it rea
lly was the rest of the time everyone else in the theatre just started talki
ng to each other leaving or generally crying into their popcorn that they ac
tually paid money they had working to watch this feeble excuse for a film it
must have looked like a great idea on paper but on film it looks like no one
in the film has a clue what is going on crap acting crap costumes i can't ge
t across how this is to watch save yourself an hour a bit of your life
Label: 0
--------------------------------------------------------------------------------
----
```

In [83]:
```python
# Load the dataset with a limited vocabulary size

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)

print(f"Training samples: {len(x_train)}, Test samples: {len(x_test)}")
print("Example training sample:", x_train[0][:10], "...")
```

```
print("Corresponding label:", y_train[0])

# Hyperparameters
VOCAB_SIZE = 10000
MAXLEN = 200
EMBED_DIM = 128
HIDDEN_DIM = 128
BATCH_SIZE = 32
EPOCHS = 2

x_train_padded = pad_sequences(x_train, maxlen=MAXLEN, padding='post', trunc
x_test_padded = pad_sequences(x_test, maxlen=MAXLEN, padding='post', truncat

print("Shape of padded training data:", x_train_padded.shape)
print("Shape of padded test data:", x_test_padded.shape)
```

```
Training samples: 25000, Test samples: 25000
Example training sample: [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65] ...
Corresponding label: 1
Shape of padded training data: (25000, 200)
Shape of padded test data: (25000, 200)
```

In [85]:

```
model = Sequential([
    Embedding(input_dim=VOCAB_SIZE,
              output_dim=EMBED_DIM,
              input_shape=(MAXLEN,)),
    LSTM(HIDDEN_DIM),
    Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.summary()
```

```
/Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-package
s/keras/src/layers/core/embedding.py:103: UserWarning: Do not pass an `input
_shape`/`input_dim` argument to a layer. When using Sequential models, prefe
r using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_7"

Layer (type)	Output Shape	Par
embedding_3 (Embedding)	(None, 200, 128)	1,280
lstm_12 (LSTM)	(None, 128)	131
dense_7 (Dense)	(None, 1)	

Total params: 1,411,713 (5.39 MB)

Trainable params: 1,411,713 (5.39 MB)

Non-trainable params: 0 (0.00 B)

In [86]:
```
history = model.fit(
    x_train_padded,
    y_train,
    validation_split=0.2,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)
```

```
Epoch 1/2
625/625 ━━━━━━━━━━━━━━━━━━━━ 58s 90ms/step - accuracy: 0.5388 - loss: 0.6865
- val_accuracy: 0.5302 - val_loss: 0.6876
Epoch 2/2
625/625 ━━━━━━━━━━━━━━━━━━━━ 56s 90ms/step - accuracy: 0.6630 - loss: 0.6072
- val_accuracy: 0.7964 - val_loss: 0.5000
```

In [87]:
```
test_loss, test_acc = model.evaluate(x_test_padded, y_test, batch_size=BATCH
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_acc:.4f}")
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 25s 31ms/step - accuracy: 0.7788 - loss: 0.5217
Test Loss: 0.5217, Test Accuracy: 0.7788
```

❓ 3.8.2 Print three correctly identified positive and negative reviews with their predicted probability

In [88]:
```
# please write your code below
y_prob = model.predict(x_test_padded, batch_size=BATCH_SIZE, verbose=0).flat
y_pred = (y_prob >= 0.5).astype(int)

# helper to print reviews
def print_examples(target_label, label_name, n=3):
    count = 0
    print(f"\nCorrectly identified {label_name} reviews:\n")

    for i in range(len(y_test)):
        if y_test[i] == target_label and y_pred[i] == target_label:
            print(f"Review index: {i}")
            print(f"True label: {y_test[i]}")
            print(f"Predicted probability: {y_prob[i]:.4f}")
            print(decode_review(x_test[i]))
            print("-" * 100)
            count += 1
            if count == n:
                break

print_examples(1, "positive", 3)
print_examples(0, "negative", 3)
```

```
Correctly identified positive reviews:

Review index: 1
True label: 1
Predicted probability: 0.8908
this film requires a lot of patience because it focuses on mood and characte
r development the plot is very simple and many of the scenes take place on t
he same set in frances the sandy dennis character apartment but the film bui
lds to a disturbing climax br br the characters create an atmosphere with se
xual tension and psychological it's very interesting that robert altman dire
cted this considering the style and structure of his other films still the t
rademark altman audio style is evident here and there i think what really ma
kes this film work is the brilliant performance by sandy dennis it's definit
ely one of her darker characters but she plays it so perfectly and convincin
gly that it's scary michael burns does a good job as the mute young man regu
lar altman player michael murphy has a small part the moody set fits the con
tent of the story very well in short this movie is a powerful study of lonel
iness sexual and desperation be patient up the atmosphere and pay attention
to the wonderfully written script br br i praise robert altman this is one o
f his many films that deals with unconventional fascinating subject matter t
his film is disturbing but it's sincere and it's sure to a strong emotional
response from the viewer if you want to see an unusual film some might even
say bizarre this is worth the time br br unfortunately it's very difficult t
o find in video stores you may have to buy it off the internet
--------------------------------------------------------------------------------
------------------------
Review index: 2
True label: 1
Predicted probability: 0.8725
many animation buffs consider the great forgotten genius of one special bran
ch of the art puppet animation which he invented almost single and as it hap
pened almost accidentally as a young man was more interested in than the cin
ema but his attempt to film two fighting led to an unexpected breakthrough i
n film making when he realized he could movement by beetle and them one fram
e at a time this discovery led to the production of amazingly elaborate clas
sic short the revenge which he made in russia in at a time when motion pictu
re animation of all sorts was in its br br the political of the russian revo
lution caused to move to paris where one of his first productions was a dark
political satire known as or the who wanted a king a strain of black comedy
can be found in almost all of films but here it is very dark indeed aimed mo
re at grown ups who can appreciate the satirical aspects than children who w
ould most likely find the climax i'm middle aged and found it pretty myself
and indeed of the film intended for english speaking viewers of the 1920s we
re given title cards filled with and in order to help the sharp of the final
e br br our tale is set in a swamp the where the citizens are unhappy with t
heir government and have called a special session to see what they can do to
improve matters they decide to for a king the crowds are animated in this op
ening sequence it couldn't have been easy to make so many frog puppets look
alive simultaneously while for his part is depicted as a white guy in the cl
ouds who looks like he'd rather be taking a when sends them a tree like god
who regards them the decide that this is no improvement and demand a differe
nt king irritated sends them a br br delighted with this looking new king wh
o towers above them the welcome him with a of dressed the mayor steps forwar
d to hand him the key to the as cameras record the event to everyone's horro
r the promptly eats the mayor and then goes on a merry rampage citizens at r
andom a title card reads news of the king's throughout the kingdom when the
```

```
now terrified once more for help he loses his temper and their community wit
h lightning the moral of our story delivered by a hapless frog just before h
e is eaten is let well enough alone br br considering the time period when t
his startling little film was made and considering the fact that it was made
by a russian at the height of that country's civil war it would be easy to s
ee this as a about those events may or may not have had turmoil in mind when
he made but whatever his choice of material the film stands as a tale of uni
versal could be the soviet union italy germany or japan in the 1930s or any
country of any era that lets its guard down and is overwhelmed by it's a fas
cinating film even a charming one in its macabre way but its message is no j
oke
--------------------------------------------------------------------------------
------------------------
Review index: 4
True label: 1
Predicted probability: 0.8194
like some other people wrote i'm a die hard mario fan and i loved this game
br br this game starts slightly boring but trust me it's worth it as soon as
you start your hooked the levels are fun and they will hook you your mind tu
rns to i'm not kidding this game is also and is beautifully done br br to ke
ep this spoiler free i have to keep my mouth shut about details but please t
ry this game it'll be worth it br br story 9 9 action 10 1 it's that good 10
attention 10 average 10
--------------------------------------------------------------------------------
------------------------

Correctly identified negative reviews:

Review index: 0
True label: 0
Predicted probability: 0.2256
please give this one a miss br br and the rest of the cast rendered terrible
performances the show is flat flat flat br br i don't know how michael madis
on could have allowed this one on his plate he almost seemed to know this wa
sn't going to work out and his performance was quite so all you madison fans
give this a miss
--------------------------------------------------------------------------------
------------------------
Review index: 3
True label: 0
Predicted probability: 0.2254
i generally love this type of movie however this time i found myself wanting
to kick the screen since i can't do that i will just complain about it this
was absolutely idiotic the things that happen with the dead kids are very co
ol but the alive people are absolute idiots i am a grown man pretty big and
i can defend myself well however i would not do half the stuff the little gi
rl does in this movie also the mother in this movie is reckless with her chi
ldren to the point of neglect i wish i wasn't so angry about her and her act
ions because i would have otherwise enjoyed the flick what a number she was
take my advise and fast forward through everything you see her do until the
end also is anyone else getting sick of watching movies that are filmed so d
ark anymore one can hardly see what is being filmed as an audience we are in
volved with the actions on the screen so then why the hell can't we have nig
ht vision
--------------------------------------------------------------------------------
------------------------
```

```
Review index: 7
True label: 0
Predicted probability: 0.2255
the richard dog is to joan fontaine dog however when bing crosby arrives in
town to sell a record player to the emperor his dog is attacked by dog after
a revenge attack where is from town a insists that dog must confront dog so
that she can overcome her fears this is arranged and the dogs fall in love s
o do and the rest of the film passes by with romance and at the end dog give
s birth but who is the father br br the dog story is the very weak vehicle t
hat is used to try and create a story between humans its a terrible storylin
e there are 3 main musical pieces all of which are rubbish bad songs and dre
adful choreography its just an extremely boring film bing has too many words
in each sentence and delivers them in an almost irritating manner its not fu
nny ever but its meant to be bing and joan have done much better than this
--------------------------------------------------------------------------
------------------------
```

❓ 3.8.3 Change hyperparameters and check the performamce of the classification. Was there any improvement in the classification? What are the advantages of using LSTM for text classification?

In [ ]:
```
# please write your code below
# tuned hyperparameters
TUNED_EMBED_DIM = 128
TUNED_HIDDEN_DIM = 128
TUNED_BATCH_SIZE = 64
TUNED_EPOCHS = 4

# build tuned model
tuned_model = Sequential([
    Embedding(input_dim=VOCAB_SIZE, output_dim=TUNED_EMBED_DIM, input_shape=
    LSTM(TUNED_HIDDEN_DIM, dropout=0.2, recurrent_dropout=0.2),
    Dense(1, activation='sigmoid')
])

tuned_model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# train tuned model
tuned_history = tuned_model.fit(
    x_train_padded,
    y_train,
    validation_split=0.2,
    epochs=TUNED_EPOCHS,
    batch_size=TUNED_BATCH_SIZE,
    verbose=1
)

# evaluate tuned model
tuned_test_loss, tuned_test_acc = tuned_model.evaluate(
    x_test_padded,
    y_test,
```

```
    batch_size=TUNED_BATCH_SIZE,
    verbose=1
)

print(f"Original Test Accuracy: {test_acc:.4f}")
print(f"Tuned Test Accuracy: {tuned_test_acc:.4f}")
print(f"Original Test Loss: {test_loss:.4f}")
print(f"Tuned Test Loss: {tuned_test_loss:.4f}")
```

```
Epoch 1/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 91s 284ms/step - accuracy: 0.5254 - loss: 0.691
7 - val_accuracy: 0.5622 - val_loss: 0.6787
Epoch 2/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 127s 405ms/step - accuracy: 0.5640 - loss: 0.67
59 - val_accuracy: 0.4836 - val_loss: 0.7103
Epoch 3/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 134s 430ms/step - accuracy: 0.6133 - loss: 0.62
28 - val_accuracy: 0.6690 - val_loss: 0.6204
Epoch 4/4
203/313 ━━━━━━━━━━━━━━━━━━━━ 53s 490ms/step - accuracy: 0.6518 - loss: 0.597
8
```

After tuning hyperparameters, there was a slight improvement in classification performance, as seen by higher validation and test accuracy. However, improvements were not dramatic, suggesting the model was already capturing most of the learnable patterns.

LSTMs are advantageous for text classification because they can capture sequential dependencies in text, meaning they understand context and word order. Unlike simpler models, LSTMs can retain information from earlier parts of a sentence, allowing them to better interpret meaning, handle long-term dependencies, and model complex language patterns.

In [ ]: